Table des matières

Labo 05 — Rendre le pipeline robuste ..... 1
Avant de commencer ..... 2
Étape 1 — Créer le paquet du pipeline ..... 2
Étape 2 — Sortir la configuration et le mot de passe du code ..... 3
Étape 3 — Journaliser, et ouvrir l'entrepôt une seule fois ..... 6
Étape 4 — Mettre les reprises en commun ..... 10
Étape 5 — Faire des scripts des étapes, et écrire la porte d'entrée ..... 13
Étape 6 — Faire tourner le pipeline, et lire ses traces ..... 24
Étape 7 — Écrire trois tests ..... 26
Étape 8 — Mettre le pipeline dans une image, et la lancer ..... 29
Étape 9 — Versionner le code, laisser dehors l'image et les données ..... 31
Dépannage ..... 32

# Labo 05 — Rendre le pipeline robuste

*Labo · Cours 5/12 — Formation Data Engineering*

But : faire des scripts des Cours 02 à 04 un paquet Python à une seule porte d'entrée, `python -m pipeline` ; sortir la configuration du code et le mot de passe du dépôt ; journaliser dans un fichier et dans une table ; mettre les reprises en commun ; écrire trois tests ; faire tourner le même pipeline dans un conteneur ; garder le tout dans le dépôt. Le pourquoi de chaque étape est dans `Theorie_05_DE.md`, section 9 et annexe A.

Trois fenêtres. Chacune reste dans son dossier du début à la fin du labo : aucun `cd` à taper après l'ouverture.

| **Fenêtre** | **Dossier** | **Ce qu'on y fait** |
|---|---|---|
| 1. Anaconda Prompt, `formations` actif | `Projet_Boreal` | le Bloc-notes, Python, `pytest`, Git |
| 2. Anaconda Prompt, à partir de l'étape 6 | `Projet_Boreal` | l'interface web de DuckDB (`duckdb -ui`) |
| 3. Anaconda Prompt devenue Ubuntu, à partir de l'étape 8 | `Projet_Boreal` | Docker |

Liste de contrôle :



© 2026 iAlexMG. Tous droits réservés.

- ☐ Paquet `pipeline` créé ; les quatre scripts des Cours 03 et 04 réécrits en modules, puis retirés par `git rm`
- ☐ Configuration lue : le port rend 5433, puis 5434 sous `PIPELINE_ERP_PORT`, toujours un entier
- ☐ Journal, entrepôt et reprises en modules communs ; les quatre scripts devenus des étapes ; la porte d'entrée écrite
- ☐ `python -m pipeline tout` sur un entrepôt effacé : 433 582 lignes de bronze, 1 572 fichiers et 513 857 lignes en `RAW`, 35 fichiers de corpus
- ☐ Échec voulu de l'étape `erp` : code de sortie 1, et la ligne « échoué » dans `pipeline.executions`
- ☐ `pytest` : `3 passed` en moins d'une seconde
- ☐ Image `boreal-pipeline` construite et lancée sur le projet monté : « à jour » partout
- ☐ Code validé dans `01_de`, `.env` absent du commit, `git status` propre

---

## Avant de commencer

Fenêtre 1, une Anaconda Prompt, dans `Projet_Boreal` :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set PYTHONPATH=01_de
REM Attendu : rien. Python cherchera aussi ses paquets dans 01_de.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir 01_de\ingestion
REM Attendu : quatre fichiers, charger_raw.py, deposer_corpus.py, extraire_erp_odbc.py et extraire_meteo_api.py.
dir 01_de\sql
REM Attendu : quatre fichiers, bronze_historien.sql, bronze_telemetrie.sql, controle_erp_base.sql et premier_regard.sql.
pytest --version
REM Attendu : pytest suivi de son numéro de version.
```

Les lignes `REM` sont des commentaires : l'Anaconda Prompt les ignore. `set PYTHONPATH=01_de` ne vaut que pour cette fenêtre : si elle se ferme, refaire `conda activate formations` et cette ligne dans la nouvelle. Si un autre résultat diffère, finir d'abord le Labo 04 ; si `pytest` manque, `pip install -r ..\01_Data_Engineering\requirements.txt`.

## Étape 1 — Créer le paquet du pipeline

*Théorie : `Theorie_05_DE.md`, annexe A.1.*



© 2026 iAlexMG. Tous droits réservés.

Un paquet est un dossier de modules Python qui s'importe sous un seul nom, comme `duckdb`, que vos scripts importent déjà. Tout le code du pipeline ira dans le paquet `pipeline` ; les scripts des Cours 03 et 04 y seront réécrits, un par un, en modules.

1.1 Créer le dossier du paquet et celui des tests. Fenêtre 1 :

```
mkdir 01_de\pipeline
mkdir 01_de\tests
REM Attendu : rien.
```

1.2 Déclarer le paquet : c'est le fichier `__init__.py` qui fait d'un dossier un paquet. Fenêtre 1 :

```
notepad 01_de\pipeline\__init__.py
```

Le Bloc-notes propose de créer le fichier : accepter. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""pipeline : le code du pipeline de Véhicules Boréal inc.

Un paquet, une porte d'entrée : python -m pipeline <étape>. Les modules de fond (config, journal,
reprise, entrepot) servent toutes les étapes ; les modules d'étape (bronze, raw, corpus, erp, meteo)
ne connaissent que leur source et exposent chacun une fonction executer(con).
"""
__version__ = "0.3.0"
```

## Étape 2 — Sortir la configuration et le mot de passe du code

*Théorie : `Theorie_05_DE.md`, annexe A.2.*

2.1 Écrire la configuration. Fenêtre 1 :

```
notepad 01_de\config.toml
```

Accepter la création. Coller, enregistrer, fermer :

```
# config.toml : tout ce que le pipeline de Boréal sait de son environnement.
#
# La configuration sort du code, mais pas les secrets : aucun mot de passe ici, ce fichier va dans
# Git. Une valeur se surcharge sans toucher au fichier, par une variable d'environnement
# PIPELINE_<SECTION>_<CLE> : PIPELINE_ERP_PORT=5434 gagne sur port = 5433 ci-dessous.
#
# Les chemins sont relatifs à Projet_Boreal, le dossier du projet, que le pipeline retrouve seul.

[chemins]
sources = "sources"
```



© 2026 iAlexMG. Tous droits réservés.

```
lac = "plateforme/lac"
entrepot = "plateforme/entrepot/boreal.duckdb"
journaux = "plateforme/journaux"

[journal]
niveau = "INFO"

[reprises]
essais = 4
attente = 2.0

[erp]
serveur = "localhost"
port = 5433
base = "erp_boreal"
utilisateur = "postgres"
paquet = 10000
tables = ["t005u", "kna1", "mara", "makt", "vbak", "vbap", "cdhdr", "cdpos", "zserie"]

[meteo]
url = "https://api.weather.gc.ca/collections/climate-daily/items"
station = "7060400"
debut = "2025-10-01"
fin = "2025-12-31"
page = 50
```

2.2 Écrire le module qui la lit, avec les secrets. Fenêtre 1 :

```
notepad 01_de\pipeline\config.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""config.py : la configuration et les secrets du pipeline, lus une fois pour toutes."""
import os
import tomllib
from functools import lru_cache
from pathlib import Path

# Le dépôt est le dossier qui porte le paquet, et il ne bouge jamais.
DEPOT = Path(__file__).resolve().parents[1]
```



© 2026 iAlexMG. Tous droits réservés.

```python
def projet() -> Path:
    """Le dossier Projet_Boreal : le parent du dépôt, sauf si PIPELINE_PROJET en désigne un autre.

    La variable déplace le projet sans déplacer le code : le conteneur s'en sert pour désigner le
    dossier monté, et les tests pour travailler dans un dossier temporaire.
    """
    return Path(os.environ.get("PIPELINE_PROJET") or DEPOT.parent)


def _surcharger(valeurs: dict) -> dict:
    """Remplace une valeur par PIPELINE_<SECTION>_<CLE> quand l'environnement en porte une.

    Le type de la valeur du fichier commande la conversion : un port reste un entier, une liste se
    coupe sur les virgules.
    """
    for section, contenu in valeurs.items():
        if not isinstance(contenu, dict):
            continue
        for cle, defaut in contenu.items():
            brut = os.environ.get(f"PIPELINE_{section.upper()}_{cle.upper()}")
            if brut is None:
                continue
            if isinstance(defaut, bool):
                contenu[cle] = brut.strip().lower() in ("1", "true", "oui")
            elif isinstance(defaut, list):
                contenu[cle] = [morceau.strip() for morceau in brut.split(",") if morceau.strip()]
            elif isinstance(defaut, (int, float)):
                contenu[cle] = type(defaut)(brut)
            else:
                contenu[cle] = brut
    return valeurs


@lru_cache(maxsize=1)
def config() -> dict:
    """La configuration du pipeline, lue une seule fois par exécution."""
    fichier = DEPOT / "config.toml"
    if not fichier.exists():
        raise FileNotFoundError(f"{fichier} est introuvable : le pipeline ne sait pas où il tourne.")
    with open(fichier, "rb") as source:
        return _surcharger(tomllib.load(source))
```



© 2026 iAlexMG. Tous droits réservés.

```
def chemin(nom: str) -> Path:
    """Un chemin de la section [chemins], rendu absolu depuis Projet_Boreal."""
    return projet() / config()["chemins"][nom]


def secret(cle: str) -> str:
    """Une valeur de 01_de/.env, un fichier de lignes CLE=valeur que Git ignore.

    L'environnement gagne sur le fichier : un conteneur reçoit ses secrets par lui, sans .env.
    """
    if cle in os.environ:
        return os.environ[cle]
    env = DEPOT / ".env"
    if env.exists():
        for ligne in env.read_text(encoding="utf-8").splitlines():
            if ligne.lstrip().startswith("#"):
                continue
            nom, _, valeur = ligne.partition("=")
            if nom.strip() == cle:
                return valeur.strip()
    raise KeyError(f"{cle} n'est ni dans l'environnement ni dans {env} : "
                   f"copiez .env.exemple en .env et remplissez-le.")
```

2.3 Lire le port de la base dans la configuration. Fenêtre 1 :

```
python -c "from pipeline.config import config; print(config()['erp']['port'])"
REM Attendu : 5433.
```

2.4 Remplacer le port par une variable d'environnement, sans toucher au fichier, pour prouver que l'environnement gagne sur la configuration. Fenêtre 1 :

```
set PIPELINE_ERP_PORT=5434
python -c "from pipeline.config import config; print(config()['erp']['port'], type(config()['erp']['port']).__name__)"
REM Attendu : 5434 int. L'environnement gagne, et le port reste un entier.
set PIPELINE_ERP_PORT=
REM Attendu : rien. La variable est retirée : le port redevient celui du fichier.
```

## Étape 3 — Journaliser, et ouvrir l'entrepôt une seule fois

*Théorie : `Theorie_05_DE.md`, annexe A.3.*

3.1 Écrire le journal : un fichier par jour, et une ligne par passage dans une table. Fenêtre 1 :



© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\pipeline\journal.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""journal.py : ce que le pipeline raconte pendant qu'il tourne, et ce qu'il en garde."""
import logging
import sys
import time
import uuid
from contextlib import contextmanager
from datetime import datetime, timezone

from . import config as conf

LOG = logging.getLogger("pipeline")


def demarrer() -> logging.Logger:
    """Branche la journalisation : la console, et un fichier par jour."""
    reglages = conf.config()["journal"]
    dossier = conf.chemin("journaux")
    dossier.mkdir(parents=True, exist_ok=True)
    fichier = dossier / f"pipeline_{datetime.now().strftime('%Y-%m-%d')}.log"

    forme = logging.Formatter("%(asctime)s %(levelname)-7s %(name)s : %(message)s",
                              datefmt="%Y-%m-%d %H:%M:%S")
    LOG.handlers.clear()
    LOG.setLevel(reglages["niveau"])
    for sortie in (logging.StreamHandler(sys.stdout),
                   logging.FileHandler(fichier, encoding="utf-8")):
        sortie.setFormatter(forme)
        LOG.addHandler(sortie)
    LOG.debug("journal ouvert dans %s", fichier)
    return LOG


def creer_table(con) -> None:
    """La table des exécutions, dans le schéma pipeline, à côté du journal des chargements."""
    con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
    con.execute("""
        CREATE TABLE IF NOT EXISTS pipeline.executions (
            execution_id VARCHAR,     -- huit caractères, pour retrouver l'exécution dans le .log
```



© 2026 iAlexMG. Tous droits réservés.

```
            etape        VARCHAR,
            debut        TIMESTAMPTZ,
            fin          TIMESTAMPTZ,
            duree_s      DOUBLE,
            etat         VARCHAR,     -- en cours, réussi, échoué
            lignes       BIGINT,      -- ce que l'étape a écrit
            message      VARCHAR)""")


class Trace:
    """Ce qu'une étape a le droit de dire d'elle-même pendant qu'elle tourne."""

    def __init__(self, execution_id: str, etape: str):
        self.execution_id = execution_id
        self.etape = etape
        self.lignes = None
        self.message = None


@contextmanager
def etape(con, nom: str):
    """Encadre une étape : une ligne dans pipeline.executions, et le temps qu'elle a pris.

    La ligne s'écrit avant le travail, à l'état « en cours » : si le processus est tué en route, une
    exécution qui n'a jamais fini se voit.
    """
    creer_table(con)
    trace = Trace(uuid.uuid4().hex[:8], nom)
    debut = datetime.now(timezone.utc)
    chrono = time.perf_counter()
    con.execute("INSERT INTO pipeline.executions VALUES (?, ?, ?, NULL, NULL, 'en cours', NULL, NULL)",
                [trace.execution_id, nom, debut])
    LOG.info("étape %s (%s) : début", nom, trace.execution_id)
    try:
        yield trace
    except BaseException as erreur:
        # La trace doit survivre à ce qui l'a interrompue. Deux précautions, dans cet ordre.
        # D'abord annuler la transaction que l'étape aurait laissée ouverte : sans cela, l'UPDATE
        # part avec elle, et la ligne reste à « en cours » en mentant. Ensuite écrire la trace sous
        # son propre try : si elle échoue à son tour, c'est la cause réelle qui doit remonter, pas
        # une erreur de journalisation.
        try:
```



© 2026 iAlexMG. Tous droits réservés.

```
            con.execute("ROLLBACK")
        except Exception:
            pass                                    # aucune transaction ouverte : rien à annuler
        try:
            _fermer(con, trace, debut, chrono, "échoué", f"{type(erreur).__name__} : {erreur}")
        except Exception as souci:
            LOG.error("étape %s (%s) : trace non écrite (%s)", nom, trace.execution_id, souci)
        LOG.error("étape %s (%s) : échec, %s", nom, trace.execution_id, erreur)
        raise
    _fermer(con, trace, debut, chrono, "réussi", trace.message)
    LOG.info("étape %s (%s) : fin, %s en %.1f s", nom, trace.execution_id,
             "aucune ligne" if trace.lignes is None else f"{trace.lignes:,} lignes".replace(",", " "),
             time.perf_counter() - chrono)


def _fermer(con, trace: Trace, debut, chrono: float, etat: str, message) -> None:
    con.execute("""
        UPDATE pipeline.executions
           SET fin = ?, duree_s = ?, etat = ?, lignes = ?, message = ?
         WHERE execution_id = ? AND debut = ?""",
              [datetime.now(timezone.utc), time.perf_counter() - chrono, etat,
               trace.lignes, message, trace.execution_id, debut])
```

3.2 Écrire la seule porte vers l'entrepôt. Fenêtre 1 :

```
notepad 01_de\pipeline\entrepot.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""entrepot.py : la seule porte vers boreal.duckdb, et le journal des chargements."""
import duckdb

from .config import chemin
from .journal import LOG


class EntrepotOccupe(RuntimeError):
    """Un autre processus tient déjà le fichier de l'entrepôt."""


def ouvrir(lecture_seule: bool = False):
    """Une connexion à l'entrepôt, ses schémas et son journal des chargements en place."""
```



© 2026 iAlexMG. Tous droits réservés.

```
    fichier = chemin("entrepot")
    fichier.parent.mkdir(parents=True, exist_ok=True)
    try:
        con = duckdb.connect(str(fichier), read_only=lecture_seule)
    except duckdb.IOException as erreur:
        raise EntrepotOccupe(
            "L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez. "
            f"({erreur})") from erreur
    LOG.debug("entrepôt ouvert : %s", fichier)
    if not lecture_seule:
        con.execute("CREATE SCHEMA IF NOT EXISTS raw")
        con.execute("CREATE SCHEMA IF NOT EXISTS pipeline")
        con.execute("""
            CREATE TABLE IF NOT EXISTS pipeline.chargements (
                fichier   VARCHAR,       -- le chemin du fichier, depuis Projet_Boreal
                empreinte VARCHAR,       -- SHA-256 du contenu
                table_raw VARCHAR,
                lot       VARCHAR,
                lignes    BIGINT,
                charge_le TIMESTAMPTZ)""")
    return con


def deja_charge(con, table: str) -> dict:
    """Ce que le journal sait de cette table : à chaque fichier, l'empreinte de sa dernière version."""
    return dict(con.execute("""
        SELECT fichier, empreinte FROM pipeline.chargements WHERE table_raw = ?
        QUALIFY ROW_NUMBER() OVER (PARTITION BY fichier ORDER BY charge_le DESC) = 1""",
                            [table]).fetchall())
```

## Étape 4 — Mettre les reprises en commun

*Théorie : `Theorie_05_DE.md`, annexe A.4.*

4.1 Écrire l'outil de reprise, qui servira à toute source qui peut faiblir. Fenêtre 1 :

```
notepad 01_de\pipeline\reprise.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""reprise.py : réessayer ce qui est passager, abandonner ce qui ne l'est pas."""
import time
```



© 2026 iAlexMG. Tous droits réservés.

```
from typing import Callable

from .config import config
from .journal import LOG


class Abandon(RuntimeError):
    """Tous les essais ont été tentés, sans succès."""


def avec_reprises(action: Callable, passagere: Callable[[Exception], bool], quoi: str,
                  essais: int = None, attente: float = None, dormir=time.sleep):
    """Lance action(), en la réessayant tant que passagere() reconnaît l'erreur.

    quoi nomme ce qu'on tentait, pour le journal. dormir s'échange dans les tests : la reprise s'y
    vérifie sans attendre quatorze secondes.
    """
    reglages = config()["reprises"]
    essais = reglages["essais"] if essais is None else essais
    attente = reglages["attente"] if attente is None else attente

    for essai in range(1, essais + 1):
        try:
            return action()
        except Exception as erreur:
            if not passagere(erreur):
                raise                                   # demande fausse : la répéter ne sert à rien
            if essai == essais:
                raise Abandon(f"{quoi} : abandon après {essais} essais ({erreur})") from erreur
            delai = attente * 2 ** (essai - 1)
            LOG.warning("%s : essai %d raté (%s), nouvel essai dans %.0f s", quoi, essai, erreur, delai)
            dormir(delai)
```

4.2 Écrire le module de l'API météo, qui remplace le script du Cours 04 : sa boucle d'essais est maintenant dans `reprise.py`. Fenêtre 1 :

```
notepad 01_de\pipeline\meteo.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""meteo.py : la vraie API d'ECCC, page par page, avec les reprises du pipeline (Cours 05).

Ce qu'il fait. Il demande à api.weather.gc.ca les observations quotidiennes de la station configurée,
```



© 2026 iAlexMG. Tous droits réservés.

```
page par page, en suivant le lien « next » que chaque réponse fournit, et écrit chaque observation
telle que reçue, une ligne JSON par jour, dans plateforme/lac/bronze/meteo_api/. À la fin, il compare
les jours reçus aux jours annoncés par l'API et refuse une extraction incomplète.

Ce que le Cours 05 a changé, par rapport au script du Cours 04. La boucle d'essais du script est
partie dans pipeline.reprise, où elle sert à toute source qui peut faiblir : le module ne garde que
ce qui lui est propre, savoir quelles erreurs HTTP valent un second essai. La station, les dates et
la taille de page viennent de la configuration, et les messages vont au journal.

La règle qu'il illustre. Ce qui se répète dans deux étapes n'appartient à aucune des deux. Une
reprise, une journalisation, une lecture de configuration sont l'affaire du pipeline ; l'étape ne
garde que la connaissance de sa source.
"""
import json
import urllib.error
import urllib.parse
import urllib.request

from .config import config, chemin
from .journal import LOG
from .reprise import avec_reprises


class ExtractionIncomplete(RuntimeError):
    """L'API a annoncé plus de jours qu'elle n'en a livré."""


def passagere(erreur: Exception) -> bool:
    """Un 429 ou un 5xx passeront peut-être ; un 4xx dit que la demande est fausse."""
    if isinstance(erreur, urllib.error.HTTPError):
        return erreur.code == 429 or erreur.code >= 500
    return isinstance(erreur, urllib.error.URLError)     # coupure, DNS, délai dépassé


def lire_page(url: str) -> dict:
    """Une page de l'API, réessayée tant que l'erreur est passagère."""
    def demander():
        with urllib.request.urlopen(url, timeout=60) as reponse:
            return json.load(reponse)
    return avec_reprises(demander, passagere, "une page de l'API météo")
```



© 2026 iAlexMG. Tous droits réservés.

```
def executer(con=None) -> int:
    """Tire les observations de la station configurée. Rend le nombre de jours reçus."""
    reglages = config()["meteo"]
    parametres = {"f": "json", "CLIMATE_IDENTIFIER": reglages["station"], "sortby": "LOCAL_DATE",
                  "datetime": f"{reglages['debut']} 00:00:00/{reglages['fin']} 00:00:00",
                  "limit": reglages["page"]}
    url = f"{reglages['url']}?{urllib.parse.urlencode(parametres)}"
    sortie = (chemin("lac") / "bronze" / "meteo_api" /
              f"quotidien_{reglages['station']}_{reglages['debut']}_{reglages['fin']}.jsonl")
    sortie.parent.mkdir(parents=True, exist_ok=True)

    pages = recus = 0
    annonces = None
    with open(sortie, "w", encoding="utf-8", newline="\n") as fichier:
        while url:
            page = lire_page(url)
            pages += 1
            annonces = page["numberMatched"]
            for entite in page["features"]:
                fichier.write(json.dumps(entite["properties"], ensure_ascii=False) + "\n")
            recus += page["numberReturned"]
            LOG.debug("  page %d : %d jours", pages, page["numberReturned"])
            # La page suivante, si l'API en annonce une.
            url = next((lien["href"] for lien in page["links"] if lien["rel"] == "next"), None)

    LOG.info("  météo : %d jours reçus sur %d annoncés, en %d pages", recus, annonces, pages)
    if recus != annonces:
        raise ExtractionIncomplete(
            f"{recus} jours reçus pour {annonces} annoncés : l'extraction est incomplète.")
    return recus
```

## Étape 5 — Faire des scripts des étapes, et écrire la porte d'entrée

*Théorie : `Theorie_05_DE.md`, annexe A.5.*

5.1 Écrire l'étape du bronze, qui lance les scripts SQL du Cours 02. Fenêtre 1 :

```
notepad 01_de\pipeline\bronze.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""bronze.py : les scripts SQL du Cours 02, lancés par le pipeline."""
```



© 2026 iAlexMG. Tous droits réservés.

```python
from pathlib import Path

from .config import DEPOT, chemin
from .journal import LOG

# Le script, et le dossier parent que son COPY partitionné attend.
SCRIPTS = [
    ("historien", "bronze_historien.sql", "bronze/usine"),
    ("télémétrie", "bronze_telemetrie.sql", "bronze/usine"),
]


class EcartDeConversion(RuntimeError):
    """La source et le bronze ne comptent pas le même nombre de lignes."""


def espacer(nombre: int) -> str:
    """275436 devient « 275 436 », comme partout ailleurs dans la formation."""
    return f"{nombre:,}".replace(",", " ")


def convertir(con, script: Path) -> tuple:
    """Exécute un script de conversion et rend son contrôle final : (lignes source, lignes bronze).

    Le script porte sa requête de contrôle en dernière instruction : c'est son résultat qu'on lit.
    """
    resultat = con.execute(script.read_text(encoding="utf-8")).fetchone()
    if resultat is None or len(resultat) != 2:
        raise EcartDeConversion(f"{script.name} ne finit pas par un contrôle à deux colonnes.")
    return int(resultat[0]), int(resultat[1])


def executer(con) -> int:
    """Refait le bronze de l'historien et de la télémétrie. Rend le total des lignes écrites."""
    total = 0
    for nom, fichier, parent in SCRIPTS:
        (chemin("lac") / parent).mkdir(parents=True, exist_ok=True)
        source, bronze = convertir(con, DEPOT / "sql" / fichier)
        if source != bronze:
            # Les nombres se mettent en forme un par un : appliquer le remplacement à la phrase
            # entière emporterait aussi sa virgule de ponctuation.
            raise EcartDeConversion(
```



© 2026 iAlexMG. Tous droits réservés.

```
                f"{nom} : {espacer(source)} lignes dans la source, {espacer(bronze)} dans le "
                "bronze. La conversion a perdu ou ajouté des lignes.")
        LOG.info("  %-12s %10s lignes, source et bronze d'accord", nom, espacer(bronze))
        total += bronze
    return total
```

5.2 Écrire le module de la `RAW`, qui remplace le chargeur du Cours 03, `charger_raw.py`. Fenêtre 1 :

```
notepad 01_de\pipeline\raw.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""raw.py : les 22 livraisons de Boréal chargées dans la couche RAW (Cours 03, empaqueté au Cours 05).

Ce qu'il fait. Pour chacune des livraisons de LIVRAISONS, il cherche les fichiers dans sources/,
calcule l'empreinte SHA-256 de chacun, et ne charge que ceux que le journal des chargements ne
connaît pas encore, ou connaît avec une autre empreinte. Chaque fichier entre tel quel dans sa table
de l'entrepôt : toutes les colonnes en texte, un objet JSON dans une colonne JSON, et trois colonnes
de lignage, _fichier, _lot et _charge_le.

La règle qu'il illustre. L'idempotence : relancer ne double rien, parce que le journal se souvient de
chaque fichier chargé. Un fichier livré de nouveau sous le même nom, avec un autre contenu, remplace
ses propres lignes : c'est ainsi qu'un export complet se met à jour. Un fichier livré sous un autre
nom est une autre livraison : l'extrait 2023-03_bis passe, et la RAW le garde, marqué par son lot.

Ce que le Cours 05 a changé, par rapport au script du Cours 03. Le module ne se lance plus seul et
n'ouvre plus l'entrepôt : il reçoit la connexion de la porte d'entrée, écrit dans le journal plutôt
qu'à l'écran, et rend ses comptes à l'appelant, qui les inscrit dans pipeline.executions. Les chemins
viennent de la configuration. La logique de chargement, elle, n'a pas bougé d'une ligne.
"""
import hashlib
from datetime import datetime, timezone
from pathlib import Path

from .entrepot import deja_charge
from .journal import LOG

# Une livraison : la table de la RAW, le motif de ses fichiers, leur forme, et d'où vient le lot.
#   dossier : le dossier du mois (2023-03, 2023-03_bis) ; jour : la date du nom de fichier ;
#   complet : un export livré en entier, qui remplace le précédent.
LIVRAISONS = [
    ("erp_t005u", "sources/erp/extraits/*/T005U.csv", "csv", "dossier"),
```



© 2026 iAlexMG. Tous droits réservés.

```
    ("erp_kna1", "sources/erp/extraits/*/KNA1.csv", "csv", "dossier"),
    ("erp_mara", "sources/erp/extraits/*/MARA.csv", "csv", "dossier"),
    ("erp_makt", "sources/erp/extraits/*/MAKT.csv", "csv", "dossier"),
    ("erp_vbak", "sources/erp/extraits/*/VBAK.csv", "csv", "dossier"),
    ("erp_vbap", "sources/erp/extraits/*/VBAP.csv", "csv", "dossier"),
    ("erp_cdhdr", "sources/erp/extraits/*/CDHDR.csv", "csv", "dossier"),
    ("erp_cdpos", "sources/erp/extraits/*/CDPOS.csv", "csv", "dossier"),
    ("erp_zserie", "sources/erp/extraits/*/ZSERIE.csv", "csv", "dossier"),
    ("garanties_clients", "sources/garanties/*/clients.csv", "csv", "dossier"),
    ("garanties_enregistrements", "sources/garanties/*/enregistrements.csv", "csv", "dossier"),
    ("garanties_visites", "sources/garanties/*/visites.csv", "csv", "dossier"),
    ("garanties_consentements", "sources/garanties/*/consentements.csv", "csv", "dossier"),
    ("garanties_reclamations", "sources/garanties/*/reclamations.json", "json", "dossier"),
    ("usine_actifs", "sources/usine/referentiel/actifs.csv", "csv", "complet"),
    ("usine_etiquettes", "sources/usine/referentiel/etiquettes.csv", "csv", "complet"),
    ("usine_bons_travail", "sources/usine/gmao/bons_travail.csv", "csv", "complet"),
    ("meteo_stations", "sources/meteo/stations.csv", "csv", "complet"),
    ("meteo_quotidien", "sources/meteo/quotidien.csv", "csv", "complet"),
    ("meteo_horaire", "sources/meteo/horaire_7060400.csv", "csv", "complet"),
    ("telemetrie_sorties", "sources/telemetrie/sorties_*.jsonl", "jsonl", "jour"),
    ("corpus_index", "sources/corpus/index.csv", "csv", "complet"),
]

# La requête qui lit une liste de fichiers, selon leur forme. Chacune rend la colonne filename.
LECTEURS = {
    "csv": "SELECT * FROM read_csv($fichiers, header = true, all_varchar = true, filename = true)",
    "json": ("SELECT json AS objet, filename "
             "FROM read_json_objects($fichiers, format = 'array', filename = true)"),
    "jsonl": ("SELECT sujet, recu_le, qos, message, filename "
              "FROM read_json($fichiers, format = 'newline_delimited', filename = true, "
              "columns = {'sujet': 'VARCHAR', 'recu_le': 'VARCHAR', 'qos': 'VARCHAR', 'message': 'JSON'})"),
}


def lot_du(fichier: str, regle: str) -> str:
    """Le lot d'un fichier : son dossier, le jour de son nom, ou « complet »."""
    chemin = Path(fichier)
    if regle == "dossier":
        return chemin.parent.name
    if regle == "jour":
        return chemin.stem.replace("sorties_", "")
    return "complet"
```



© 2026 iAlexMG. Tous droits réservés.

```
def empreinte(fichier: str) -> str:
    """L'empreinte SHA-256 du contenu : deux fichiers identiques la partagent, un octet la change."""
    return hashlib.sha256(Path(fichier).read_bytes()).hexdigest()


def a_charger(con, table: str, motif: str, regle: str) -> list:
    """Les fichiers du motif que le journal ne connaît pas, ou connaît sous une autre empreinte.

    Les chemins se lisent depuis Projet_Boreal, où la porte d'entrée s'est placée : le lignage de la
    RAW s'écrit alors de la même façon sous Windows et sous Linux.
    """
    connus = deja_charge(con, table)
    fichiers = sorted(p.as_posix() for p in Path(".").glob(motif))
    return [(f, sceau, lot_du(f, regle)) for f in fichiers
            if connus.get(f) != (sceau := empreinte(f))]


def charger_une(con, table: str, motif: str, forme: str, regle: str) -> tuple:
    """Charge les fichiers nouveaux ou modifiés d'une livraison. Rend (fichiers, lignes)."""
    nouveaux = a_charger(con, table, motif, regle)
    if not nouveaux:
        return 0, 0

    source = LECTEURS[forme]
    noms = [f for f, _, _ in nouveaux]
    charge_le = datetime.now(timezone.utc)
    con.execute("BEGIN")
    try:
        # La table naît de la forme du premier fichier : ses colonnes, en texte, puis le lignage.
        con.execute(f"""
            CREATE TABLE IF NOT EXISTS raw.{table} AS
            SELECT * EXCLUDE (filename), filename AS _fichier, '' AS _lot, NOW() AS _charge_le
            FROM ({source}) LIMIT 0""", {"fichiers": noms[:1]})
        con.execute("CREATE OR REPLACE TEMP TABLE a_charger "
                    "(fichier VARCHAR, empreinte VARCHAR, lot VARCHAR)")
        con.executemany("INSERT INTO a_charger VALUES (?, ?, ?)", nouveaux)
        # Un fichier déjà chargé sous une autre empreinte : ses anciennes lignes partent d'abord.
        con.execute(f"DELETE FROM raw.{table} WHERE _fichier IN (SELECT fichier FROM a_charger)")
        con.execute(f"""
            INSERT INTO raw.{table}
```



© 2026 iAlexMG. Tous droits réservés.

```
            SELECT s.* EXCLUDE (filename), s.filename, a.lot, $charge_le
            FROM ({source}) s JOIN a_charger a ON a.fichier = s.filename""",
                    {"fichiers": noms, "charge_le": charge_le})
        con.execute(f"""
            INSERT INTO pipeline.chargements
            SELECT a.fichier, a.empreinte, '{table}', a.lot, COUNT(r._fichier), $charge_le
            FROM a_charger a LEFT JOIN raw.{table} r
              ON r._fichier = a.fichier AND r._charge_le = $charge_le
            GROUP BY ALL""", {"charge_le": charge_le})
        lignes = con.execute(f"SELECT COUNT(*) FROM raw.{table} WHERE _charge_le = ?",
                             [charge_le]).fetchone()[0]
        con.execute("COMMIT")
    except Exception:
        # Qui ouvre la transaction la referme. Sans ce ROLLBACK, DuckDB la garde avortée, et la
        # requête suivante, y compris celle qui ferme la ligne de l'étape, répond « Current
        # transaction is aborted » au lieu de laisser passer la vraie cause.
        con.execute("ROLLBACK")
        raise
    LOG.debug("%s : %d fichier(s), %d lignes", table, len(noms), lignes)
    return len(noms), lignes


def executer(con) -> int:
    """Charge les 22 livraisons. Rend le nombre de lignes chargées à ce passage.

    Comme toute étape du pipeline, elle rend un entier : c'est ce que la porte d'entrée inscrit dans
    la colonne lignes de pipeline.executions. Le compte des fichiers, lui, va au journal de texte.
    """
    total_fichiers = total_lignes = 0
    for table, motif, forme, regle in LIVRAISONS:
        fichiers, lignes = charger_une(con, table, motif, forme, regle)
        if fichiers:
            LOG.info("  %-26s %4d fichier(s), %8s lignes", table, fichiers,
                     f"{lignes:,}".replace(",", " "))
        else:
            LOG.info("  %-26s    à jour", table)
        total_fichiers += fichiers
        total_lignes += lignes
    LOG.info("  RAW : %s fichier(s) chargé(s) à ce passage", f"{total_fichiers:,}".replace(",", " "))
    return total_lignes
```

5.3 Écrire le module de la base de l'ERP, qui remplace l'extraction ODBC du Cours 04, `extraire_erp_odbc.py`. Fenêtre 1 :



© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\pipeline\erp.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""erp.py : les neuf tables de la base de l'ERP, lues par ODBC (Cours 04, empaqueté au Cours 05).

Ce qu'il fait. Il se connecte à la base de l'ERP de Boréal, dans son conteneur PostgreSQL, par le
pilote psqlODBC, lit chaque table du schéma sap par paquets, l'écrit dans un CSV temporaire, puis
DuckDB la range en Parquet dans plateforme/lac/bronze/erp_base/.

La règle qu'il illustre. Une dépendance qui ne sert qu'à une étape s'importe dans cette étape, pas en
tête du paquet. pyodbc et le pilote psqlODBC ne s'installent que pour cette extraction : le reste du
pipeline, celui que rejoue le kit 02, n'importe que DuckDB et la bibliothèque standard, et tourne
même quand pyodbc n'est pas là. Le mot de passe, lui, n'est ni dans le code ni dans config.toml : il
est lu par config.secret, dans 01_de/.env ou dans l'environnement du conteneur.

Lancement : python -m pipeline erp, le conteneur de la base démarré.
"""
import csv
import time

import duckdb

from .config import config, chemin, secret
from .journal import LOG


class PiloteAbsent(RuntimeError):
    """Aucun pilote ODBC de PostgreSQL n'est installé sur ce poste."""


def chaine_de_connexion(pyodbc) -> str:
    """La chaîne ODBC, pilote compris. Changer de base ne change que cette chaîne."""
    pilotes = [p for p in pyodbc.drivers() if p.startswith("PostgreSQL Unicode")]
    if not pilotes:
        raise PiloteAbsent(f"Pilote psqlODBC introuvable. Pilotes ODBC installés : {pyodbc.drivers()}")
    reglages = config()["erp"]
    return (f"DRIVER={{{pilotes[0]}}};SERVER={reglages['serveur']};PORT={reglages['port']};"
            f"DATABASE={reglages['base']};UID={reglages['utilisateur']};"
            f"PWD={secret('ERP_MOT_DE_PASSE')}")
```



© 2026 iAlexMG. Tous droits réservés.

```
def executer(con=None) -> int:
    """Extrait les neuf tables de la base. Rend le total des lignes lues."""
    import pyodbc                                   # dépendance de cette étape seulement

    reglages = config()["erp"]
    sortie = chemin("lac") / "bronze" / "erp_base"
    sortie.mkdir(parents=True, exist_ok=True)

    debut = time.perf_counter()
    total = 0
    connexion = pyodbc.connect(chaine_de_connexion(pyodbc))
    curseur = connexion.cursor()
    duck = duckdb.connect()                         # une base en mémoire : l'entrepôt n'est pas touché
    for table in reglages["tables"]:
        curseur.execute(f"SELECT * FROM sap.{table}")
        colonnes = [c[0].upper() for c in curseur.description]
        temporaire = sortie / f"{table}.csv"
        lignes = 0
        with open(temporaire, "w", newline="", encoding="utf-8") as fichier:
            ecrivain = csv.writer(fichier)
            ecrivain.writerow(colonnes)
            while paquet := curseur.fetchmany(reglages["paquet"]):
                ecrivain.writerows(paquet)
                lignes += len(paquet)
        # Le bronze garde le texte reçu ; DuckDB le range en Parquet compressé.
        duck.execute(f"""
            COPY (SELECT * FROM read_csv('{temporaire.as_posix()}', header = true, all_varchar = true))
            TO '{(sortie / f"{table}.parquet").as_posix()}' (FORMAT parquet, COMPRESSION zstd)""")
        temporaire.unlink()
        LOG.info("  sap.%-8s %8s lignes", table, f"{lignes:,}".replace(",", " "))
        total += lignes
    connexion.close()
    LOG.info("  neuf tables extraites en %.1f s", time.perf_counter() - debut)
    return total
```

5.4 Écrire le module du corpus, qui remplace le dépôt du Cours 03, `deposer_corpus.py`. Fenêtre 1 :

```
notepad 01_de\pipeline\corpus.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""corpus.py : la documentation de Boréal déposée telle quelle dans le bronze (Cours 03, empaqueté au 05).
```



© 2026 iAlexMG. Tous droits réservés.

```
Ce qu'il fait. Il copie chaque fichier de sources/corpus/ dans plateforme/lac/bronze/corpus/, en
gardant l'arborescence, et saute ceux dont l'empreinte n'a pas bougé. Le catalogue, lui, entre dans
la RAW comme une table, par l'étape raw.

La règle qu'il illustre. Tout ne se met pas en table. Un document se dépose tel quel, avec sa date et
son empreinte, et c'est ce dépôt que la formation AI ira lire. Déposer reste une étape du pipeline :
elle se journalise, se relance sans dégât, et rend un compte.
"""
import hashlib
import shutil

from .config import chemin
from .journal import LOG


def empreinte(fichier) -> str:
    return hashlib.sha256(fichier.read_bytes()).hexdigest()


def executer(con=None) -> int:
    """Dépose le corpus dans le bronze. Rend le nombre de fichiers déposés à ce passage."""
    source = chemin("sources") / "corpus"
    bronze = chemin("lac") / "bronze" / "corpus"
    deposes = identiques = 0
    for fichier in sorted(p for p in source.rglob("*") if p.is_file()):
        cible = bronze / fichier.relative_to(source)
        if cible.exists() and empreinte(cible) == empreinte(fichier):
            identiques += 1
            continue
        cible.parent.mkdir(parents=True, exist_ok=True)
        # copyfile, et non copy2 : recopier les métadonnées échoue quand un conteneur Linux écrit
        # sur un montage de fichiers Windows, « [Errno 1] Operation not permitted », et l'étape n'en
        # a pas besoin. C'est l'empreinte du contenu qui dit si un document a bougé, pas sa date.
        shutil.copyfile(fichier, cible)
        deposes += 1
    LOG.info("  corpus : %d fichier(s) déposé(s), %d déjà à jour", deposes, identiques)
    return deposes
```

5.5 Retirer les quatre scripts d'origine, que les modules remplacent. Git les efface du disque et note leur retrait pour le prochain commit. Fenêtre 1 :

```
git -C 01_de rm -r ingestion
REM Attendu : quatre lignes, de rm 'ingestion/charger_raw.py' à rm 'ingestion/extraire_meteo_api.py'.
```



© 2026 iAlexMG. Tous droits réservés.

5.6 Compter les étapes : chaque module d'étape expose une fonction `executer`. Fenêtre 1 :

```
findstr /R /C:"^def executer" 01_de\pipeline\*.py
REM Attendu : cinq lignes, une par module : bronze, corpus, erp, meteo et raw.
```

5.7 Écrire la porte d'entrée. Fenêtre 1 :

```
notepad 01_de\pipeline\__main__.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""__main__.py : la porte d'entrée du pipeline de Boréal.

    python -m pipeline bronze raw corpus     les étapes nommées, dans cet ordre
    python -m pipeline tout                  les étapes du chemin hors ligne, dans le bon ordre
    python -m pipeline raw --bavard          le journal descend au niveau DEBUG
    python -m pipeline --etapes              la liste des étapes, sans rien lancer
"""
import argparse
import os
import sys

from . import bronze, corpus, erp, meteo, raw
from .config import projet
from .entrepot import ouvrir
from .journal import LOG, demarrer, etape

# Le nom de l'étape en ligne de commande, ce qu'elle fait, et si « tout » la prend.
ETAPES = {
    "bronze": (bronze.executer, "les CSV et le JSON de l'usine, rangés en Parquet", True),
    "raw": (raw.executer, "les 22 livraisons chargées dans la couche RAW", True),
    "corpus": (corpus.executer, "la documentation déposée telle quelle dans le bronze", True),
    "erp": (erp.executer, "les neuf tables de la base de l'ERP, par ODBC", False),
    "meteo": (meteo.executer, "les observations d'ECCC, par son API", False),
}


def lire_arguments(arguments=None) -> argparse.Namespace:
    analyseur = argparse.ArgumentParser(
        prog="python -m pipeline",
        description="Le pipeline de Véhicules Boréal inc. : une étape, plusieurs, ou tout.")
    analyseur.add_argument("etapes", nargs="*", metavar="ÉTAPE",
```



© 2026 iAlexMG. Tous droits réservés.

```
                          help="les étapes à lancer, ou « tout »")
    analyseur.add_argument("--bavard", action="store_true", help="journal au niveau DEBUG")
    analyseur.add_argument("--etapes", dest="lister", action="store_true",
                          help="liste les étapes et ce qu'elles font, sans rien lancer")
    return analyseur.parse_args(arguments)


def resoudre(demandees: list) -> list:
    """La liste des étapes à lancer, dans l'ordre. « tout » prend celles du chemin hors ligne."""
    if demandees == ["tout"]:
        return [nom for nom, (_, _, dans_tout) in ETAPES.items() if dans_tout]
    inconnues = [nom for nom in demandees if nom not in ETAPES]
    if inconnues:
        raise SystemExit(f"Étape inconnue : {', '.join(inconnues)}. "
                         f"Connues : {', '.join(ETAPES)}, ou « tout ».")
    return demandees


def main(arguments=None) -> int:
    options = lire_arguments(arguments)
    if options.lister or not options.etapes:
        print("Étapes du pipeline :")
        for nom, (_, quoi, dans_tout) in ETAPES.items():
            print(f"  {nom:8} {quoi}{'' if dans_tout else '   (hors de « tout »)'}")
        return 0

    a_lancer = resoudre(options.etapes)
    # Le pipeline travaille depuis Projet_Boreal, d'où qu'on l'appelle : les chemins des scripts SQL
    # et le lignage de la RAW s'écrivent alors toujours de la même façon.
    os.chdir(projet())
    if options.bavard:
        os.environ["PIPELINE_JOURNAL_NIVEAU"] = "DEBUG"
    demarrer()
    LOG.info("pipeline : %s, depuis %s", ", ".join(a_lancer), projet())

    con = ouvrir()
    try:
        for nom in a_lancer:
            action = ETAPES[nom][0]
            with etape(con, nom) as trace:
                # Une étape rend un entier, ce qu'elle a écrit. Le convertir ici fait dire à une
                # étape mal écrite ce qu'elle a fait de travers, plutôt qu'à DuckDB deux lignes
```



© 2026 iAlexMG. Tous droits réservés.

```
                # plus loin, dans une erreur de conversion de type.
                resultat = action(con)
                trace.lignes = None if resultat is None else int(resultat)
        except Exception as erreur:
            LOG.error("pipeline arrêté : %s", erreur)
            return 1
        finally:
            con.close()
        LOG.info("pipeline : terminé")
        return 0


    if __name__ == "__main__":
        sys.stdout.reconfigure(encoding="utf-8", errors="replace")
        sys.exit(main())
```

## Étape 6 — Faire tourner le pipeline, et lire ses traces

*Théorie : `Theorie_05_DE.md`, annexe A.6.*

6.1 Voir les étapes du pipeline, sans rien lancer. Fenêtre 1 :

```
python -m pipeline --etapes
REM Attendu : cinq étapes, bronze, raw, corpus, erp et meteo ; les deux dernières (hors de « tout »).
```

6.2 Lancer tout le chemin hors ligne, sur l'état que les Labos 02 à 04 ont laissé. Fenêtre 1 :

```
python -m pipeline tout
REM Attendu : historien 275 436 et télémétrie 158 146 lignes, source et bronze d'accord ;
REM puis à jour sur les 22 livraisons, 35 fichiers de corpus déjà à jour, et pipeline : terminé.
```

6.3 Effacer ce que le pipeline reconstruit depuis les sources, et cela seulement : l'entrepôt et le corpus déposé. Fenêtre 1 :

```
rmdir /s /q plateforme\entrepot
rmdir /s /q plateforme\lac\bronze\corpus
REM Attendu : rien.
```

6.4 Relancer tout, pour voir un premier passage complet. Fenêtre 1 :

```
python -m pipeline tout
REM Attendu : le bronze comme en 6.2 ; les 22 livraisons chargées, de erp_t005u 49 fichier(s)
REM à corpus_index 1 fichier(s), puis RAW : 1 572 fichier(s) chargé(s) à ce passage ;
REM corpus : 35 fichier(s) déposé(s), 0 déjà à jour ; pipeline : terminé.
```



© 2026 iAlexMG. Tous droits réservés.

Les chiffres à retrouver :

| Étape | Ce qu'elle écrit | Durée chez l'auteur |
|---|---|---|
| `bronze` | 433 582 lignes (275 436 et 158 146) | 7,1 s |
| `raw` | 1 572 fichiers, 513 857 lignes, 22 tables | 13,2 s |
| `corpus` | 35 fichiers déposés | moins de 0,1 s |

6.5 Erreur voulue : lancer l'étape `erp` sur un port où aucune base n'écoute, puis lire le code de sortie. Fenêtre 1 :

```
set PIPELINE_ERP_PORT=5439
python -m pipeline erp
REM Attendu : après environ 4 s, étape erp (...) : échec, ('08001', ... port 5439 failed: Connection refused ...),
REM puis pipeline arrêté, suivi du même message.
echo %ERRORLEVEL%
REM Attendu : 1.
set PIPELINE_ERP_PORT=
REM Attendu : rien. Le port redevient 5433.
```

À retenir — un échec se dit par le code de sortie. `%ERRORLEVEL%` rend le code que le programme a laissé en finissant : 0 quand tout est passé, 1 ici. C'est la seule chose qu'un ordonnanceur lit avec certitude ; un pipeline qui écrirait « échec » dans son journal et rendrait 0 serait compté comme réussi. On provoque l'échec par la configuration, sans rien casser : la base de l'ERP, elle, n'a pas bougé.

6.6 Voir le journal de texte, un fichier par jour. Fenêtre 1 :

```
dir plateforme\journaux
REM Attendu : un fichier, pipeline_ suivi de la date du jour, .log.
```

6.7 Ouvrir l'interface web de DuckDB. Fenêtre 2 : une nouvelle Anaconda Prompt, dans `Projet_Boreal`, puis :

```
conda activate formations
duckdb -ui
```

Laisser cette fenêtre ouverte : la fermer arrête l'interface. Créer un carnet neuf (+ à côté de Notebooks) et le renommer « Labo 05 » (double-clic sur son titre, puis Entrée). Une requête par cellule, Ctrl+Entrée pour l'exécuter.

6.8 Attacher l'entrepôt, en lecture seule.

```
-- Attendu : 0 rows returned.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
```

6.9 Lire la table des exécutions, le passage le plus récent en dernier.



© 2026 iAlexMG. Tous droits réservés.

```
-- Attendu : 4 lignes. bronze, raw et corpus réussi, avec 433582, 513857 et 35 lignes ;
-- erp échoué, sans lignes, avec le début du message de l'erreur.
SELECT etape, etat, lignes, ROUND(duree_s, 1) AS duree_s, LEFT(message, 40) AS message
FROM entrepot.pipeline.executions
ORDER BY debut;
```

6.10 Détacher l'entrepôt : l'étape 8 écrira dedans.

```
-- Attendu : 0 rows returned.
DETACH entrepot;
```

Point d'arrêt. Le pipeline est écrit, il tourne, et ses passages sont dans la table. Qui fait ce labo en deux séances s'arrête ici : les étapes 7 à 9 ajoutent les tests, l'image et le commit, et ne supposent rien de plus que le dépôt tel qu'il est maintenant. À la reprise, refaire « Avant de commencer » dans la fenêtre 1.

## Étape 7 — Écrire trois tests

*Théorie : `Theorie_05_DE.md`, annexe A.7.*

7.1 Dire à `pytest` où chercher les tests et le paquet. Fenêtre 1 :

```
notepad 01_de\pytest.ini
```

Accepter la création. Coller, enregistrer, fermer :

```
# pytest.ini : où pytest cherche les tests, et où il cherche le paquet.
#
# Sans pythonpath, « pytest » lancé depuis 01_de s'arrête sur « ModuleNotFoundError: No module named
# 'pipeline' », alors que « python -m pytest » passe : la seconde forme ajoute le dossier courant aux
# chemins d'importation, la première non. Ce fichier fait que les deux marchent.

[pytest]
pythonpath = .
testpaths = tests
```

7.2 Écrire les trois tests. Fenêtre 1 :

```
notepad 01_de\tests\test_pipeline.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""test_pipeline.py : trois propriétés du pipeline, vérifiées sans les données de Boréal."""
import urllib.error
```



© 2026 iAlexMG. Tous droits réservés.

```python
import pytest

from pipeline import bronze, raw
from pipeline.config import config
from pipeline.entrepot import ouvrir
from pipeline.meteo import passagere
from pipeline.reprise import Abandon, avec_reprises


@pytest.fixture
def projet(tmp_path, monkeypatch):
    """Un Projet_Boreal jetable : le pipeline y écrit son entrepôt et ses fichiers."""
    monkeypatch.setenv("PIPELINE_PROJET", str(tmp_path))
    config.cache_clear()
    monkeypatch.chdir(tmp_path)
    yield tmp_path
    config.cache_clear()


def ecrire_extrait(projet, mois: str, lignes: list) -> None:
    dossier = projet / "sources" / "erp" / "extraits" / mois
    dossier.mkdir(parents=True, exist_ok=True)
    contenu = "VBELN,NETWR\n" + "".join(f"{v}\n" for v in lignes)
    (dossier / "VBAK.csv").write_text(contenu, encoding="utf-8")


def test_le_journal_refuse_un_extrait_deja_charge(projet):
    """Recharger le même fichier ne double rien ; le même fichier corrigé remplace ses lignes."""
    ecrire_extrait(projet, "2023-01", ["0000000001,100", "0000000002,200"])
    con = ouvrir()
    try:
        motif, table = "sources/erp/extraits/*/VBAK.csv", "erp_vbak"

        fichiers, lignes = raw.charger_une(con, table, motif, "csv", "dossier")
        assert (fichiers, lignes) == (1, 2)

        # Deuxième passage, rien n'a changé : le journal reconnaît l'empreinte et ne recharge rien.
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (0, 0)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 2

        # Le même nom, un autre contenu : ses anciennes lignes partent, les nouvelles entrent.
        ecrire_extrait(projet, "2023-01", ["0000000001,100", "0000000002,200", "0000000003,300"])
```



© 2026 iAlexMG. Tous droits réservés.

```
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (1, 3)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 3

        # Un autre nom de dossier est une autre livraison : le journal la laisse passer, et la RAW
        # garde les deux lots. C'est le piège de 2023-03_bis, et il est voulu.
        ecrire_extrait(projet, "2023-01_bis", ["0000000001,100"])
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (1, 1)
        assert con.execute(f"SELECT COUNT(DISTINCT _lot) FROM raw.{table}").fetchone()[0] == 2
    finally:
        con.close()


def test_la_conversion_garde_le_compte_des_lignes(projet, tmp_path):
    """Le contrôle d'un script de conversion rend le même compte des deux côtés, ou il lève."""
    source = projet / "mesures"
    source.mkdir()
    for jour, valeurs in (("2025-10-01", [1.0, 2.0]), ("2025-10-02", [3.0])):
        (source / f"mesures_{jour}.csv").write_text(
            "Tag,Value\n" + "".join(f"UTL.CMP2.VIBRATION,{v}\n" for v in valeurs), encoding="utf-8")

    script = tmp_path / "conversion.sql"
    script.write_text("""
        COPY (
            SELECT Tag, Value,
                   CAST(REPLACE(PARSE_FILENAME(filename, true), 'mesures_', '') AS DATE) AS jour
            FROM read_csv('mesures/*.csv', header = true, filename = true,
                          columns = {'Tag': 'VARCHAR', 'Value': 'DOUBLE'})
        ) TO 'bronze/mesures' (FORMAT parquet, PARTITION_BY (jour), OVERWRITE);
        SELECT (SELECT COUNT(*) FROM 'mesures/*.csv') AS lignes_csv,
               (SELECT COUNT(*) FROM 'bronze/mesures/*/*.parquet') AS lignes_parquet;
        """, encoding="utf-8")
    (projet / "bronze").mkdir()

    con = ouvrir()
    try:
        assert bronze.convertir(con, script) == (3, 3)

        # Un script qui ne finit pas par un contrôle à deux colonnes est refusé, plutôt que cru.
        muet = tmp_path / "muet.sql"
        muet.write_text("SELECT 1;", encoding="utf-8")
        with pytest.raises(bronze.EcartDeConversion):
            bronze.convertir(con, muet)
```



© 2026 iAlexMG. Tous droits réservés.

```
    finally:
        con.close()


def test_la_reprise_s_arrete_apres_le_nombre_d_essais_prevu(projet):
    """Quatre essais, trois attentes qui doublent, puis l'abandon. Une erreur définitive ne passe pas."""
    essais = []
    attentes = []

    def toujours_en_panne():
        essais.append(1)
        raise urllib.error.HTTPError("https://exemple", 503, "Service Unavailable", None, None)

    with pytest.raises(Abandon):
        avec_reprises(toujours_en_panne, passagere, "essai", essais=4, attente=2.0,
                      dormir=attentes.append)
    assert len(essais) == 4
    assert attentes == [2.0, 4.0, 8.0]

    # Un 404 dit que la demande est fausse : une seule tentative, et l'erreur remonte telle quelle.
    definitive = []

    def demande_fausse():
        definitive.append(1)
        raise urllib.error.HTTPError("https://exemple", 404, "Not Found", None, None)

    with pytest.raises(urllib.error.HTTPError):
        avec_reprises(demande_fausse, passagere, "essai", essais=4, attente=2.0,
                      dormir=attentes.append)
    assert len(definitive) == 1
```

7.3 Lancer les tests. Fenêtre 1 :

```
pytest 01_de
REM Attendu : collected 3 items, puis 3 passed, en moins d'une seconde (0,3 s chez l'auteur).
```

## Étape 8 — Mettre le pipeline dans une image, et la lancer

*Théorie : `Theorie_05_DE.md`, annexe A.8.*

8.1 Écrire la recette de l'image. Fenêtre 1 :

```
notepad 01_de\Dockerfile
```



© 2026 iAlexMG. Tous droits réservés.

Accepter la création. Coller, enregistrer, fermer :

```
# Dockerfile : le pipeline de Boréal, empaqueté dans une image.
#
# L'image fige Python, DuckDB et le code. Elle ne contient ni données ni secrets : le projet arrive
# par un montage, le mot de passe par une variable d'environnement.

FROM python:3.11-slim

# La version est fixée : une image qu'on reconstruit dans six mois donne le même moteur SQL.
RUN pip install --no-cache-dir duckdb==1.5.5

WORKDIR /app/01_de
COPY pipeline/ pipeline/
COPY sql/ sql/
COPY config.toml ./

# Le projet n'est pas dans l'image : il arrive par un montage, à cet endroit.
ENV PIPELINE_PROJET=/travail
ENV PYTHONUNBUFFERED=1

ENTRYPOINT ["python", "-m", "pipeline"]
CMD ["--etapes"]
```

8.2 Vérifier le nom du fichier : le Bloc-notes ajoute `.txt` à un nom sans extension. Fenêtre 1 :

```
dir 01_de\Dockerfile*
REM Attendu : un seul fichier, Dockerfile. S'il s'appelle Dockerfile.txt : ren 01_de\Dockerfile.txt Dockerfile
```

8.3 Dire à Docker ce que la construction ne doit même pas voir. Fenêtre 1 :

```
notepad 01_de\.dockerignore
```

Accepter la création. Coller, enregistrer, fermer :

```
# .dockerignore : ce que la construction de l'image ne doit même pas voir.
#
# Docker envoie le dossier courant au moteur avant de construire. Ce qui est listé ici ne part pas,
# donc ne peut pas se retrouver dans l'image par accident.

# Le secret ne voyage pas dans une image : il arrive par l'environnement, au lancement.
.env

# Les tests appartiennent à l'atelier, pas au pipeline livré.
```



© 2026 iAlexMG. Tous droits réservés.

```
tests/
pytest.ini

__pycache__/
.pytest_cache/
```

8.4 Ouvrir Ubuntu. Fenêtre 3 : une nouvelle Anaconda Prompt, dans `Projet_Boreal`, puis :

```
wsl
```

Attendu : une invite qui finit par `…/Projet_Boreal$`. Cette fenêtre est désormais Ubuntu, dans le dossier du projet, et y reste.

8.5 Construire l'image à partir du dépôt. Fenêtre 3 :

```
docker build -t boreal-pipeline 01_de
# Attendu : naming to docker.io/library/boreal-pipeline:latest. La première fois, le téléchargement
# de Python et de DuckDB prend de quinze secondes à une minute ; ensuite, deux secondes.
```

8.6 Lancer les étapes `raw` et `corpus` dans un conteneur, sur le projet monté. Fenêtre 3 :

```
docker run --rm -v "$PWD":/travail boreal-pipeline raw corpus
# Attendu : à jour sur les 22 livraisons, RAW : 0 fichier(s) chargé(s) à ce passage,
# corpus : 0 fichier(s) déposé(s), 35 déjà à jour, puis pipeline : terminé.
```

Le conteneur a lu l'entrepôt que Windows avait écrit en 6.4, et reconnu chaque fichier : rien à recharger. L'étape `raw` y prend 6 à 7 s, contre moins d'une demi-seconde sous Windows pour le même travail.

## Étape 9 — Versionner le code, laisser dehors l'image et les données

*Théorie : `Theorie_05_DE.md`, annexe A.9.*

Fenêtre 1. `git -C 01_de` lance Git dans le dépôt sans quitter `Projet_Boreal`, et les chemins qui suivent partent de `01_de` :

```
git -C 01_de add pipeline tests config.toml pytest.ini Dockerfile .dockerignore
REM Attendu : rien.
git -C 01_de status
REM Attendu : Changes to be committed, 15 fichiers en new file, trois scripts d'ingestion/ en deleted,
REM et renamed: ingestion/charger_raw.py -> pipeline/raw.py ; aucun .env.
git -C 01_de commit -m "Cours 05 : le pipeline en paquet, configuré, journalisé, testé et conteneurisé"
REM Attendu : 19 files changed, dont rename ingestion/charger_raw.py => pipeline/raw.py (57%).
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `exit` dans la fenêtre 3, `.exit` à l'invite `D` de la fenêtre 2, puis fermer l'onglet du navigateur.



© 2026 iAlexMG. Tous droits réservés.

## Dépannage

| Symptôme | Correctif |
|---|---|
| `No module named pipeline`, ou `ModuleNotFoundError: No module named 'pipeline'` | la fenêtre 1 n'a pas son `set PYTHONPATH=01_de` (fenêtre rouverte) : le retaper, puis relancer |
| `ModuleNotFoundError: No module named 'duckdb'` | l'environnement `formations` n'est pas actif : `conda activate formations`, puis relancer |
| Le Bloc-notes ouvre un fichier qui a déjà du contenu, en 4.2 ou de 5.2 à 5.4 | un reste d'un premier essai : tout sélectionner (Ctrl+A), coller le module à sa place, enregistrer |
| `git rm` répond `pathspec 'ingestion' did not match any files` | les scripts d'origine sont déjà retirés : passer à la suite |
| `git rm` répond `Permission denied` sans coupable visible | un verrou passager de Windows : fermer les éditeurs et terminaux qui regardent ces fichiers, puis réessayer |
| `AttributeError: module 'pipeline.raw' has no attribute 'executer'` | le module n'a pas été collé en entier : le recoller, puis effacer `01_de\pipeline\__pycache__` |
| `FileNotFoundError: …config.toml est introuvable` | `config.toml` n'est pas dans `01_de`, à côté du dossier `pipeline` : le déplacer |
| `L'entrepôt est occupé : fermez la ligne de commande DuckDB, puis relancez.` | l'interface tient l'entrepôt : `DETACH entrepot;` dans le carnet, ou `.exit` dans la fenêtre 2, puis relancer |
| `rmdir` répond que le fichier est utilisé par un autre processus | l'interface ou un pipeline tient l'entrepôt : `DETACH entrepot;` dans le carnet, puis refaire 6.3 |
| `bronze : … lignes dans la source, … dans le bronze` | un script SQL du Cours 02 a été modifié : le comparer à celui du Labo 02 |
| `bronze_historien.sql ne finit pas par un contrôle à deux colonnes` | la requête de contrôle du script a disparu : la remettre en dernière instruction, comme au Labo 02 |
| `Conversion Error: Unimplemented type for cast` à la fin d'une étape | l'étape rend autre chose qu'un entier : chaque `executer` rend un seul nombre, celui des lignes écrites |
| `SyntaxError` sur une ligne comme `pythonpath = .` dans `test_pipeline.py` | le contenu de `pytest.ini` a été collé dans le fichier de tests : ne garder que ce qui suit `# -*- coding: utf-8 -*-` |



© 2026 iAlexMG. Tous droits réservés.

| Symptôme | Correctif |
|---|---|
| `pytest` ne trouve aucun test (`no tests ran`) | `pytest` lancé sans le dossier : `pytest 01_de` |
| `failed to read dockerfile: open Dockerfile: no such file or directory` | le fichier s'appelle `Dockerfile.txt` : `ren 01_de\Dockerfile.txt Dockerfile` dans la fenêtre 1 |
| `docker: permission denied while trying to connect` | votre compte Ubuntu n'est pas dans le groupe `docker` : l'étape 4 du guide d'installation, `sudo usermod -aG docker $USER`, puis rouvrir Ubuntu |
| Le conteneur recharge les 1 572 fichiers | le lignage a été écrit en chemins absolus ou à barres inversées : vérifier que `a_charger` de `raw.py` liste les fichiers par `Path(".").glob(motif)` et `as_posix()` |
| `pytest` passe alors que le code est cassé exprès | c'est le test qu'il faut corriger, pas le code |



© 2026 iAlexMG. Tous droits réservés.